Unit 4 – Database Connectivity with Python

Database

-A database is defined as a collection of logically related data stored together that is designed to meet the information needs of an organization.
-It can contain one data file (a very small database) or large number of data files (a large database) depending on organizational needs.
-A database is organized in such a way that a computer program can quickly select desired pieces of data.
-A database can be of any size and of varying complexity.
-A database is a systematic collection of data.
-They support electronic storage and manipulation of data.
-Databases make data management easy.

Database Management System (DBMS)

-The database and DBMS software together is called a database system.
-Database Management system (DBMS) is a collection of programs that enables users to create and maintain database.
-DBMS is basically a computerized record-keeping system; which stores information and allows users to add, delete, change, retrieve and update that information on demand.
-It provides for simultaneous use of a database by multiple users and tool for accessing and manipulating the data in the database.
-A database system overcomes the limitations of traditional file-oriented system such as, large amount of data redundancy, poor data control, inadequate data manipulation capabilities and excessive programming effort by supporting an integrated and centralized data structure.
-Operations Performed on Database Systems:

1. Inserting new data into existing table 2. Adding new tables to the database.

Advantages of DBMS

a). Minimal data redundancy:
-In a database system, views of different user groups (data files) are integrated during database design into a single, logical, centralized structure.
-Each primary fact is ideally recorded in only one place in the database. The total data storage requirement is effectively reduced.
-Incidentally, it does not mean or suggest that all redundancy can or necessarily should be eliminated.

b). Efficient data access:
-DBMS utilizes a variety of sophisticated techniques to store and retrieve data efficiently.
-This feature is especially important if the data is stored on external storage devices.

c). Improved data sharing:
-Since, database system is a centralized repository of data belonging to the entire organization it can be shared by all authorized users.
-Existing application programs can share the data in the database. Furthermore, new application programs can be developed on the existing data in the database to share the same data and add only that data that is not currently stored, rather having to define all data requirements again

d). Improved data consistency:
-In database system, such inconsistencies are avoided to some extent by making them known to DBMS.

-DBMS ensures that any change made to either of automatically applied to the other one as well.

e). Improved data integrity:
-Integrity is usually expressed in terms of constraints, which are consistency rules that the database system should not violate.
-Another integrity check can be incorporated in the database to ensure that if there is reference to a certain object, that object must exist.
-For example, in the case of bank's automatic teller machine (ATM), a user is not allowed to transfer fund from a non-existent saving to a checking account.

f). Improved security:
-Database security is the protection of database from unauthorized users.
-The database administrator (DBA) ensures that proper access procedure is followed, including proper authentication schemes for access to the DBMS and additional checks before permitting access to sensitive data.
-Different levels of security could be implemented for various types of data and operations.
-The access of data by authorized user may be restricted for each type of access (for example, retrieve, insert, modify, update, delete and so on) to each piece of information in the database.

g). *Reduced program maintenance:*
-The problems of high maintenance effort required in file-oriented system, are reduced in database system.
-In a file-oriented environment, the descriptions of data and the logic for accessing data are built into individual application programs.
-As a result, changes to data formats and access methods inevitably result in the need to modify application programs.
In database environment, data are more independent of the application programs.

h). *Improved backup and recovery services:*
-DBMS provides facilities for recovering from hardware or software failures through its back up and recovery subsystem.
-For example, if the computer system fails in the middle of a complex update program, the recovery subsystem is responsible and makes sure that the database is restored to the state it was in before the program started executing.
-Alternatively, the recovery subsystem ensures that the program is resumed from the point at which it was interrupted so that its full effect is recorded in the database.

SQL fundamentals
Create Table: The CREATE TABLE statement is used to create a new table in a database.
Syntax:

```
CREATE TABLE table_name (
                                column1 datatype,
                                column2 datatype,
                                column3 datatype,
                                ....
);
```

Insert: The INSERT INTO statement is used to insert new records in a table.
Syntax:

```
INSERT INTO table_name (column1, column2, column3,              ...)
VALUES (value1, value2, value3, ...);
```

Update: The UPDATE statement is used to modify the existing records in a table.
Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2,                          ...
WHERE condition;
```

Delete: The DELETE statement is used to delete existing records in a table.
Syntax:

```
DELETE FROM table_name WHERE condition;
```

Select: The SELECT statement is used to display the data of a table or selective columns data from the table.
SELECT * FROM table_name
SELECT column_name1, column_name2, column_namen FROM table_name

Python Database Connectivity
MySql with Python

```
#Checking the connection
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password=""
# )
# print(mydb)

#Creating the database
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password=""
# )
# # print(mydb)
# mycursor = mydb.cursor()
# mycursor.execute("CREATE DATABASE mypy")

#Show the list of databases
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password=""
# )
# mycursor = mydb.cursor()
# mycursor.execute("SHOW DATABASES")
```

```
# for x in mycursor:
#   print(x)

#Creating the table stud_data in the database mypy
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password="",
#   database="mypy"
# )
# mycursor = mydb.cursor()
# mycursor.execute("CREATE TABLE stud_data (roll INT, name VARCHAR(25))")

#Inserting the data
# import mysql.connector
# try:
#   connection = mysql.connector.connect(
#       host='localhost',
#       database='mypy',
#       user='root',
#       password='')

#   mySql_insert_query = "INSERT INTO stud_data (roll,name) VALUES (%s, %s) "
#   records_to_insert = [(1, 'Hello'),(2,'Sunday'),(3,'Monday')]
#   cursor = connection.cursor()
#   cursor.executemany(mySql_insert_query, records_to_insert)
#   connection.commit()
#   print(cursor.rowcount, "Record inserted successfully into Stud table")
# except mysql.connector.Error as error:
#   print("Failed to insert record into MySQL table {}".format(error))
# finally:
#   if connection.is_connected():
#       cursor.close()
#       connection.close()
#       print("MySQL connection is closed")


#To display all the records from the table
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password="",
#   database="mypy"
# )
# mycursor = mydb.cursor()
# mycursor.execute("SELECT * FROM stud_data")
# myresult = mycursor.fetchall()
# for x in myresult:
#   print(x)
```

```
#Update
# import mysql.connector
# mydb = mysql.connector.connect(
#   host="localhost",
#   user="root",
#   password="",
#   database="my_database_python"
# )
# mycursor = mydb.cursor()
# sql = "UPDATE Stud SET address = 'Pune' WHERE name = 'Priya Dave'"
# mycursor.execute(sql)
# mydb.commit()
# print(mycursor.rowcount, "record(s) affected")

#Delete
import mysql.connector
mydb = mysql.connector.connect(
   host="localhost",
   user="yourusername",
   password="yourpassword",
   database="mjkacc"
)
mycursor = mydb.cursor()
sql = "DELETE FROM Stud WHERE id = '5'"
mycursor.execute(sql)
mydb.commit()
print(mycursor.rowcount, "record(s) deleted")
```

Sqlite with Python

```
#Create database
import sqlite3
cnt=sqlite3.connect('atmiya.dp')

#Creating the table stud_data
# cnt.execute('create table stud_data(roll integer, name varchar)')
# print('Table created')

#Inserting the records
# cnt.execute("""
#                    insert into stud_data values
#                    (1,'January'),
#                    (2,'February'),
#                    (3,'March')
#          """)
# print('Record Inserted Successfully')
# cnt.commit()

#Retriving the data
# cursor=cnt.execute('select * from stud_data')
# for i in cursor:
#          print(i)
```

```
#Retriving the record whose roll no greater than 2
# cursor=cnt.execute('select * from stud_data where roll>2')
# for i in cursor:
#	print(i)

#Update the name whose roll no is 3
# sql_upd="""update stud_data set name='April' where roll=3"""
# cnt.execute(sql_upd)
# cursor=cnt.execute('select * from stud_data')
# for i in cursor:
#	print(i)

#Deleting the record whose roll is 3
sql_del="""delete from stud_data where roll=3"""
cnt.execute(sql_del)
cursor=cnt.execute('select * from stud_data')
for i in cursor:
	print(i)
```